

Kristálytiszta Elektronika 2

A Kristálytiszta Elektronika tananyag fejlesztését az Európai Bizottság támogatta az Erasmus+ program keretén belül „2020-1-HU01-KA201-078779” számú „Elektronika és programozás témájú innovatív tananyag fejlesztése a köznevelés számára” című projekt kapcsán.

Az Európai Unió
Erasmus+ programjának
társfinanszírozásával



A projektet az alábbi 4 intézményből álló nemzetközi partnerség valósította meg:

- Xtalin Mérnöki Tervező Kft. – Budapest
- ELTE Bolyai János Gyakorló Általános Iskola és Gimnázium – Szombathely
- Bolyai Farkas Elméleti Líceum – Târgu Mureș
- Pro Ratio Alapítvánnyal karöltve működő Madách Imre Gimnázium – Šamorín



XTALIN



Szerzői jogok:

Jelen tananyag az Xtalin Mérnöki Tervező Kft., mint koordinátor által vezetett partnerség szellemi terméke. Az anyagok oktatási felhasználásra készültek, ezért e célra szabadon felhasználhatóak; tartalmuk azonban nem módosítható, továbbfejleszthető az Xtalin Kft. írásban adott engedélye nélkül. Az anyagok újraközlése tartalmilag változatlan formában, kizárólag a tananyag szerzőinek és az eredeti tananyag elérhetőségének világos megjelölésével lehetséges, kizárólag az Xtalin Kft. írásban adott engedélyével.

Kapcsolat: <http://kristalytisztaelektronika2.hu/>
info@kristalytisztaelektronika.hu

8. Soros kommunikációs vonalak I.

Írta: Szabó Béla

Lektorálta: Hadarics Szabolcs

Kedves Olvasó!

Ezen tananyagrészen szeretném bemutatni Neked a soros kommunikációs vonalak közül az I²C és az SPI buszokat. A többiről, a teljesség igénye nélkül: UART/USART, CAN, USB, egy másik tananyagrészen olvashatsz.

ADATÁTVITEL

ADATÁTVITELI TÍPUSOK

A kommunikációs vonalaink tervezésekor előre tudnunk kell, hogy milyen irányba szeretnénk adatot továbbítani. Ezek a következők lehetnek:

- Szimplex (angolul: simplex)
- Fél-duplex (angolul: half-duplex)
- Teljes-duplex (angolul: full-duplex)

A simplex adatátvitel során az adat csak egy irányba haladhat. Tehát egy olyan hálózatról beszélünk, ahol az adó és vevő szerepek állandóak.

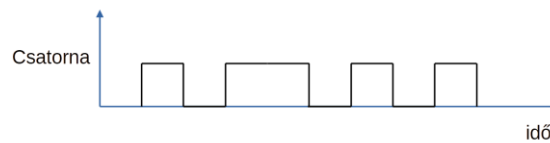
A half-duplex kommunikáció közben az adó és vevő irányok az adott kommunikációs ciklus végén megváltozhatnak, tehát az adatfolyamunk két irányú lehet. Fontos megjegyezni, hogy egy időpillanatban csak egy irányba történik az adatáramlás.

A full-duplex adattovábbítás alatt a kommunikációban résztvevő mindkét oldal egyszerre képes adatot küldeni a másinak.

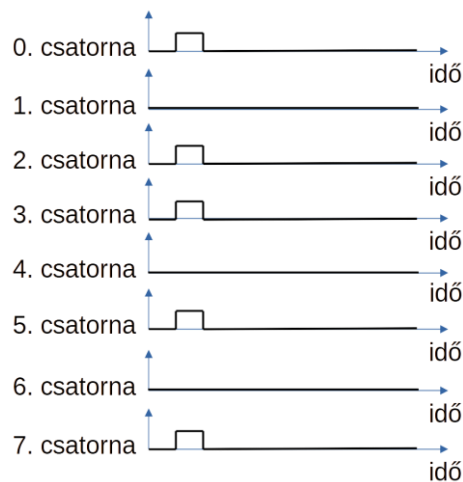
SOROS ÉS PÁRHUZAMOS KOMMUNIKÁCIÓS VONAL ÖSSZEHASONLÍTÁSA

Még mielőtt ténylegesen belemegyünk a mélyébe, először is szeretném kifejtetni, hogy mit is jelent a “soros kommunikációs vonal” és “párhuzamos kommunikációs vonal”. A következő egyszerű kis példával hasonlítjuk össze őket.

Tegyük fel, hogy szimplex kommunikációval szeretnénk 1 bájtnyi (8 bitnyi) információt továbbítani. A soros kommunikációban az adatunk bitjei egymást követve haladnak végig a két eszközünk között, míg a párhuzamos kommunikáció esetén egy időben több bitet vagyunk képesek küldeni és a másik oldalon olvasni. A soros kommunikáció emiatt több időbe, a párhuzamos pedig több kommunikációs lábba kerül. De a dolgunk sajnos, mint már tudjuk, nem ilyen egyszerű. Plusz “költségek” előfordulhatnak. Az I²C és az SPI esetében majd be is mutatok ilyeneket. A soros és párhuzamos megvalósításokat az alábbi két ábra szemlélteti.



1. ábra - A soros kommunikáció adatátvittele



2. ábra - A párhuzamos kommunikáció adatátvittele

Mivel a beágyazott rendszerekben viszonylag ritkán alkalmazunk párhuzamos kommunikációs protokollokat, ezért a tananyag további részében a soros kommunikációs formákat szeretném bemutatni. Térjünk vissza az 1. ábrához!

A képet alaposabban szemügyre véve azt vehetjük észre, hogy bizonyos ideig magasabb, majd alacsonyabb feszültségszinten van a jelünk. A magasabb szint fogja nekünk a logikai 1 szintet jelölni, míg az alacsonyabb a logikai 0 értéket képviseli. Ebből a két állapotból azt láthatjuk, hogy ez egy kétállapotú (bináris) adatfolyam. Mindegyik bit ugyanolyan széles (a vízszintes tengely az időt jelöli, tehát ugyanolyan hosszú ideig tart minden egyes bit). Számozzuk meg az adatbitekét! (Már valószínűleg megtanultad egy korábbi tananyagrészből, hogy a beágyazott programozás során nullától kezdjük a számozást.) A nulladik és hetedik bitek különleges elnevezést is kapnak, nem csak a számukkal tudunk rájuk hivatkozni. Itt ebben az esetben legyen a nulladik bit a legkisebb helyiértékű bit (least significant bit, rövidebben LSB), míg a hetedik bit a legnagyobb helyiértékű (most significant bit, vagyis MSB).

Kitekintés:

A helyiértékek sorrendje történelmi viták alapja. Általánosságban elmondható, hogy az úgynevezett bájtrend (vagy esetünkben bitsorrend) egyszerűen csak konvenció, hogy az egyes alkalmazási területeken melyik sorrendet használjuk. Azt a sorrendet, ahol az alacsonyabb helyiérték kerül hamarabb feldolgozásra little endian bitsorrendnek (vagy bájtrendnek) nevezzük, ha viszont a bitek (vagy bájtok) fordított sorrendben követik egymást akkor a neve big endian.

Ez a konvenció nem csak kommunikációs csatornák esetében kerül elő, hanem a memóriakezelés

esetén is fontos szerepet játszik. A nevek eredete két nagy gyártó, az Intel és a Motorola nevéhez fűződik, akik a saját processzoraik memóriaszervezésében ellentétes bájtrendet követtek.

Mindig figyeljünk rá, hogy a little vagy big endian kifejezés jelenthet bit- és bájtrendet is jelenthet úgy, mint az LSB és MSB is egyaránt vonatkozhat bitekre és bájtokra is. Fontos, hogy amikor ezeket a kifejezéseket használjuk, mindig pontosan legyünk tisztában a szövegkörnyezettel, ugyanis könnyű félreértelmezni, miről is van szó.

Az előbb elmondottak alapján kérlek próbáld megfejteni azt, hogy milyen értéket látsz az ábrán (segíthet a számítógépen lévő számológép "programozói mód"-ja). A megoldást (kérlek még ne nézd meg) a következő oldal tetején, dőlt betűvel találod.



Az általunk mutatott adatbájti: kettes számrendszerben: 0b10101101; tízes számrendszerben: 173; tizenhatos számrendszerben 0xAD.

I²C

Az I²C (Inter-Integrated Circuit) alapjában véve egy master-slave kommunikációt valósít meg, továbbá a multimaster megvalósítás is lehetséges. A master-slave kommunikáció azt jelenti, hogy az adott buszon csak a master eszköz kezdeményezhet kommunikációt, a slave eszköz(ök) az ő kéréseit teljesíti(k). A multimaster buszon bármely master kezdeményezheti a kommunikációt, de csak akkor, ha a buszon nincs adatforgalom. A továbbiakban csak a master-slave topológiával foglalkozunk, mivel az az elterjedtebb.

Rögtön hozok is egy példát. Talán a kezdetben is tartod a tananyag fejlesztői panelét. Ezen a panelen található egy STM32F405 mikrovezérlő, különféle szenzorok, és pár csatlakozó is, melyek között van egy olyan, amivel a kijelzőt tudod csatlakoztatni a panelen létrehozott I²C buszra. A szenzorok közül jelenleg nekünk a fénymérő, a hőmérő és a páratartalom-mérő az érdekes, mivel a nyomtatott áramkörön (NYÁK) a vezérlővel együtt ez a négy eszköz szerepel az I²C vonalakon. Ezek közül csak a mikrovezérlő kezdeményezhet kommunikációt, a többieknek a nekik szóló utasításokat kell végrehajtani. Ilyen utasítás például az, hogy egy szenzor adja meg a mért értékét, vagy hogy a kijelző jelenítse meg azt az általunk meghatározott módon.

I²C CÍMZÉS

Az előbb megemlítettem azt, hogy a slave eszközök csak azokat az utasításokat hajtják végre, amik nekik szólnak. Mindegyik eszköznek van egy saját címe, amelyre várja az utasításokat. Az eszközök címét a gyártók határozzák meg, de vannak olyan eszközök, amelyeknek van kivezetve címzőlábak, amelyek segítségével tudunk módosítani ezen. A lehetséges címeket a gyártók az alkatrészek adatlapjában mindig meghatározzák. Az általunk használt fénymérő IC rendelkezik az előbb megemlített címmódosító lábbal, ami segítségével a 3. ábrán szereplő címeket veheti fel.

ADDR SEL Terminal Level	Slave Address
GND	0101001
Float	0111001
V _{DD}	1001001

3. ábra - A fénymérő szenzor lehetséges címei

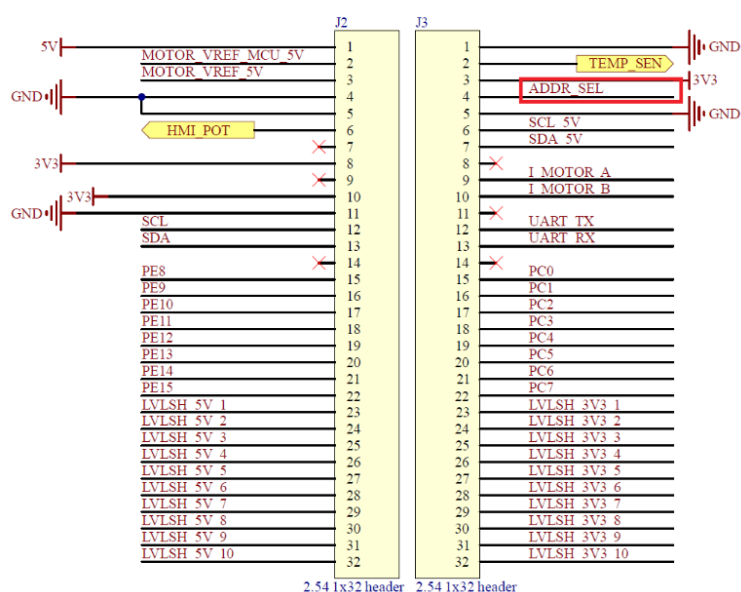
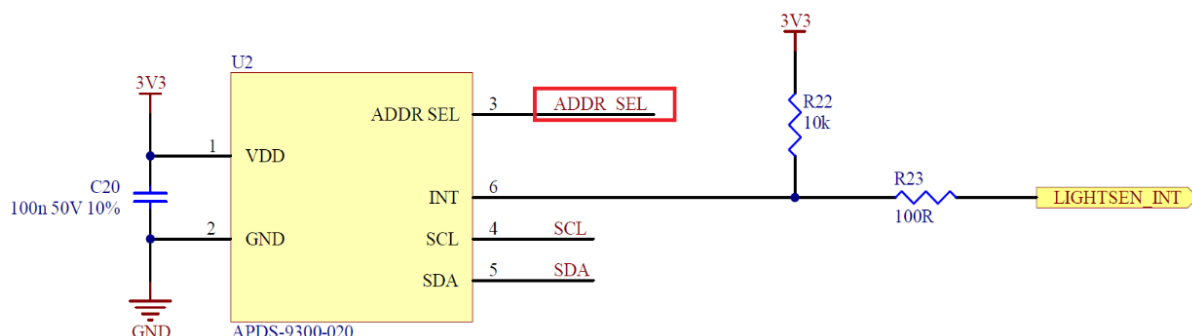
Forrás: Broadcom

<https://docs.broadcom.com/doc/AV02-1077EN>

Nézzük végig, hogy milyen feszültség szinteket vehet fel a láb! A GND jelöli a földet, a V_{DD} pedig a tápfeszültséget. Ezeken felül, ha nem kötjük sehova, tehát a láb "lebeg", akkor azt is egy érvényes állapotnak értelmezi az eszköz, és ennek megfelelően állítja be a címét.

Ha jobban megnézed a címeket, láthatod, hogy hét bit szélesek. Általában a kommunikáció legelterjedtebb alapegysége a bájt. Ez itt sincs másképp, csak egy kis trükk kell a nyolcadik bithez. Ez határozza meg azt, hogy a master írni vagy olvasni szeretne-e a slave eszközről. Az íráshoz nullát, míg az olvasáshoz egyet kell beállítanunk.

Hogy lássuk is, mire gondolok, itt egy újabb példa. A fénymérő által mért értéket szeretnénk lekérdezni. A fénymérő címzésért felelős ADDR_SEL lábát (4. ábra) most kössük a földre (GND)! Ekkor az eszköz a 0b0101001_x címre fog figyelni. Az aláhúzott bit helyére, mivel olvasni szeretnénk a szenzor egyik regiszterét, 1-et kell írunk. A teljes címzés tehát 0b01010011 (0x53).



Az ADDR_SEL jel a fejlesztői panel ADDRSEL feliratú lábára csatlakozik.

A cím beállításához ezt a jelet 3V3-ra, GND-re lehet kapcsolni, illetve lebegve hagyni

4. ábra - A fénymérő szenzor bekötése a fejlesztői panelen

A SZENZOR I²C REGISZTEREI

Ebben a tananyagrészen azt tapasztalhattad eddig, hogy szépen, fokozatosan közelítünk a teljesség felé, úgy, hogy elhintek egy-egy szót, amiből következtethetsz a következő lépésünkre. Az utolsó ilyen az volt, hogy *“mivel olvasni szeretnénk a szenzor egyik regiszterét, 1-et kell írunk”*. A legtöbb szenzornak, így az általunk használt fénymérőnek is több regisztere van. Ilyenek például a státusz-, a vezérlő- és az adatregiszterek.

- A státuszregiszterek általában az integrált áramkör (Integrated Circuit, továbbiakban IC) állapotjelző bitjeit tartalmazzák. Ilyen jelzőbit lehet például a mérés elkészülte, vagy, hogy az IC működése engedélyezve van-e.
- A vezérlőregiszterekkel befolyásolhatjuk az IC viselkedését, például a mérési gyakoriságot és a megszakítások küldését.
- Az adatregiszterekben tárolják a szenzorok az elkészült mérési eredményeket.

Nézzük meg konkrétan, hogy milyen, számunkra érdekes regiszterei vannak a fénymérő szenzornak! Ehhez keressük meg az adatlapjának a 9. oldalát, vagy csak nézzük meg az alábbi táblázatot. Ez a táblázat az adatlapban meghatározott regiszterek címét (hexadecimálisan), nevét (angolul) és egy egyszerű magyar nyelvű leírást tartalmaz arról, hogy az adott regisztert mire lehet használni.

Regiszter címe (hexadecimálisan)	Regiszter neve	Regiszter leírása
--	COMMAND	Címző regiszter, tárolja a műveletben résztvevő belső regiszter címét
0x00	CONTROL	Alapfunkciókat vezérlő regiszter
0x01	TIMING	Integrálási időt és erősítést vezérlő regiszter
0x02	THRESHLOWLOW	Alacsony megszakítási határértékhez tartozó alsó bájt
0x03	THRESHLOWHIGH	Alacsony megszakítási határértékhez tartozó felső bájt
0x04	THRESHHIGHLOW	Magas megszakítási határértékhez tartozó alsó bájt
0x05	THRESHHIGHHIGH	Magas megszakítási határértékhez tartozó felső bájt
0x06	INTERRUPT	Az interrupt beállításokat tároló regiszter
0x0A	ID	Az IC azonosítóját és verziószámát tartalmazó regiszter
0x0C	DATA0LOW	A nulladik ADC-csatornának az alsó adatbájta
0x0D	DATA0HIGH	A nulladik ADC-csatornának a felső adatbájta
0x0E	DATA1LOW	Az első ADC-csatornának az alsó adatbájta
0x0F	DATA1HIGH	Az első ADC-csatornának a felső adatbájta

1. táblázat - A fényerősség mérő IC regiszterei

Kitekintés:

Elsőre megütközhetünk rajta, hogy a COMMAND regiszter nem rendelkezik címmel. Erre azonban egyszerű magyarázatot kaphatunk, ha az eszköz kezelését végig gondoljuk. Az I2C-n kapott első bájtot a szenzor mindig "command"-nak értelmezi. Ez a "command" bájt határozza meg, hogy melyik belső regisztert fogjuk használni, de ezt el is kell tárolnunk a szenzorban, amíg tart a kommunikáció. Pontosan erre való a COMMAND regiszter, eltárolja az I2C transfer első adatbájtját.

I²C ÁTVITELI SZEKVENCIÁK

Oké, oké úgy érzem, hogy most jogos lehet a következő kérdés:

"Már tudjuk, hogy mit lehet írni és olvasni a fénymérőből, de ezt hogyan tudjuk megtenni?"

Erre a választ szintén a fénymérő adatlapjából fejteném ki, de először itt egy magyarázó tábla, hogy mi mit fog jelenteni.

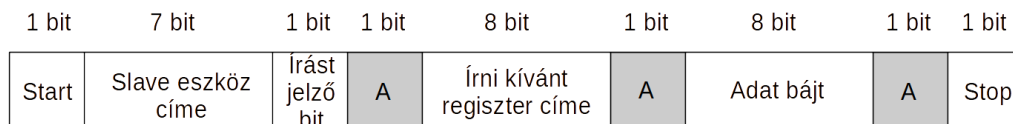
Jelölés	Leírás
A	Nyugtázó bit (ACK - Acknowledge, NACK - Not Acknowledge)
P	Stop bit
Rd	Olvasás bit, aminek az értéke 1
S	Start bit
Sr	Ismételt start bit
Wr	Írás bit, aminek az értéke 0
...	Az előző szekvencia folytatása
Fehér háttér	Adatirány: Master a Slave felé
Szürke háttér	Adatirány: Slave a Master felé

2. táblázat - I²C kommunikációs ábrák jelölései

Általános esetben az I²C kommunikáció egy címezésből, és utána tetszőleges mennyiségű adatbájt írásából vagy olvasásából áll. Minden kommunikációt a master kezdeményez egy slave cím buszra

hajtásával. A kommunikáció menete alkalmazásspecifikus, hiszen a slave egységek kialakításától függ, milyen művelethez hány és milyen bájtot kell küldenie vagy fogadnia a master egységnek.

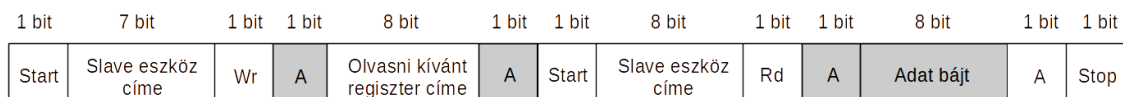
Nézzük meg, hogyan írhatunk a slave eszköz egy regiszterébe! A következőkben egy ilyen írási folyamatot szeretnék bemutatni, amit a következő ábra is illusztrál.



5. ábra - I²C egy bájt írásának szekvenciája

1. A folyamat azzal kezdődik, hogy a master jelzi a slave felé, hogy most kezdődik egy adás. Ezt egy Start bittel teszi meg, ami egy speciális bit, pont ugyanúgy, mint a Stop bit. Ezeket egy kicsit később fejtem ki.
2. A master a Start bitet követően a slave eszköz 7 bites címét, majd az írási szándékot jelző 0 értékű bitet továbbítja. Ezt követően, ha van ilyen címmel elérhető eszköz a buszon, akkor ő ezt az érkezett bájtot nyugtázza (ACK bitet küld). A nyugtázást szintén később fogom részletezni.
3. Az ACK után az írni kívánt regiszter címe (azaz a command code) kerül, amit a slave szintén nyugtáz.
4. A következő küldött bájt már a tényleges adatunk lesz, amit szeretnénk a slave eszközben beállítani. Mint az előbbi két bájtot, ezt is nyugtázza a slave.
5. A master a továbbiakban nem szeretne több adatot írni, így kiküldi a már említett Stop bitet.

Most nézzünk meg egy olvasási folyamatot! Az olvasás eleje megegyezik az írási folyamattal. Ezt az alábbi ábrán láthatod.

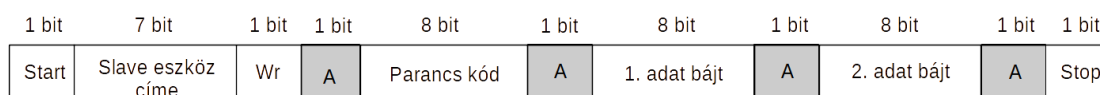


6. ábra - I²C egy bájt olvasásának szekvenciája

1. A master egy Start bitet küld.
2. Kiküldi a slave eszköz címét, majd utána az írást jelző 0 bitet. Ha jól címeztük meg az eszközünk, akkor nyugtázza a vételt.
3. A master kiküldi az olvasni kívánt regiszter címét, majd várja az ACK-ot.
4. Miután a visszajelzés megérkezett, újra kell indítanunk a kommunikációt azért, hogy írási módból olvasásra váltsunk. Ehhez küldünk egy újabb Start bitet.
5. A Start bit után újra küldjük a Slave eszköz címét és most, mivel olvasni akarunk adatot, így az olvasási 1-es bitet küldjük.
6. Megvárunk egy ACK-ot és az után már a slave eszköz küldi a kívánt adatot.
7. Miután az utolsó adatbit is beérkezett a bájtból, most a master eszköz ad visszajelzést a slave-nek. Mivel már nem szeretne több adatot küldeni, ezért a master egy NACK-ot fog küldeni. Ezt is egy kicsit később, az ACK kifejtésénél fogom elmagyarázni.

8. Az utolsó lépés pedig az, hogy a master befejezze a kommunikációt egy Stop bittel.

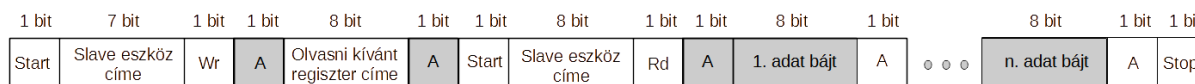
Van lehetőségünk a buszon egymást követő értékeket írni és olvasni. Ebben az esetben egy címbájt írását követően egy command code tartalmával jelezzük a slave eszköz felé, hogy több adatot szeretnénk átvinni a buszon (ez lehet egy sokbites mért érték olvasása, de akár egy konfiguráció feltöltése is). Ezt követően jelennek meg az adatbájtok a buszon, amiket a fogadó fél ACK jelekkel nyugtáz.



7. ábra - I²C két bájt írásának szekvenciája

Hasonlítsuk össze először az egy és két adatbájtos írás műveletét! Mindkét szekvencia ugyanúgy néz ki az első írt adatbájt küldéséig. A különbség itt kezdődik. A második bájt előtt a master már nem a Stop bitet, hanem - a slave által küldött ACK után - a következő regiszternek szánt adatot küldi. Ezt addig folytatja a master, ameddig szeretne írni. Az utolsó írni kívánt bájt után még megvárja az ACK-ot és kiküldi a Stop bitet, ezzel lezárva a folyamatot.

A több bájtos olvasás során is jó kiindulási alapot ad nekünk az egy bájt olvasásának ismerete. Miután a slave elküldte az első adatbájtot, a master nem NACK-ot küld a slave-nek, hanem ACK-kal nyugtázza az üzenetet, egészen az utolsó venni kívánt bájtig. Ekkor a master az egy bájtos olvasásnál is használt NACK-ot küldi a slave-nek, aki emiatt már nem küld több bájtot, így a master lezárja a kommunikációt a Stop bittel.

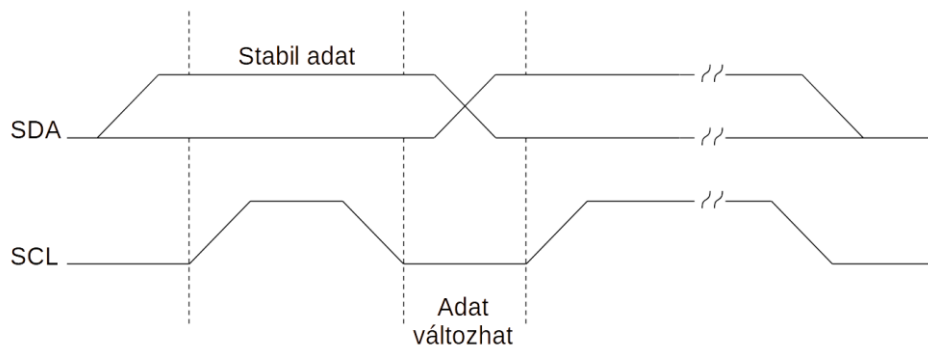


8. ábra - I²C bájtok folyamatos olvasásának szekvenciája

I²C JELEK, JELALAKOK

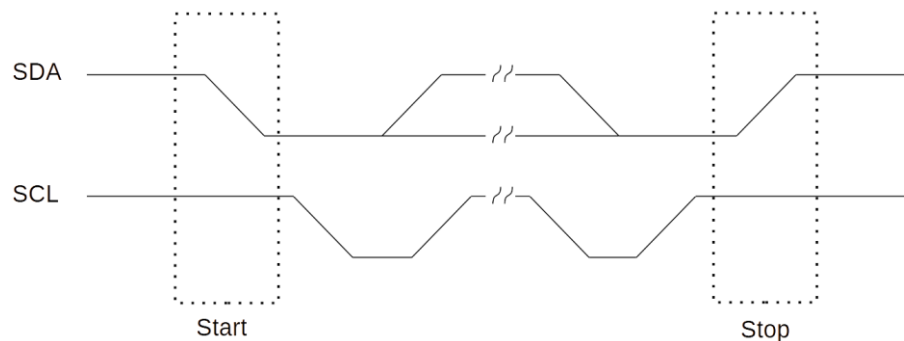
Na de mit is jelent egy bit átküldése az I²C buszon?

Az I²C egy kétvezetékes, szinkron kommunikáció. A szinkronitás azt jelenti, hogy van egy vezérlő órajel, amivel szinkronban történik az adatáramlás. Az előbb említett két vezeték az SDA és az SCL. Az SDA (Serial Data, soros adat) lábán történik egy adatbit küldése vagy fogadása, amit az SCL (Serial Clock, soros órajel) ütemével fogunk írni vagy olvasni. Ezt a 9. ábrán látható jelalakok szemléltetik. Itt azt látjuk, hogy az érvényes adatbithez az SDA lábat még az SCL láb magas szintre állítása előtt be kell állítani a kívánt értékre, majd stabilan azon kell tartani, amíg az SCL láb szintje újra alacsony nem lesz. Ebből az következik, hogy adatbit esetén szigorúan csak akkor változhat az SDA láb szintje, amikor az SCL láb alacsony szinten van.



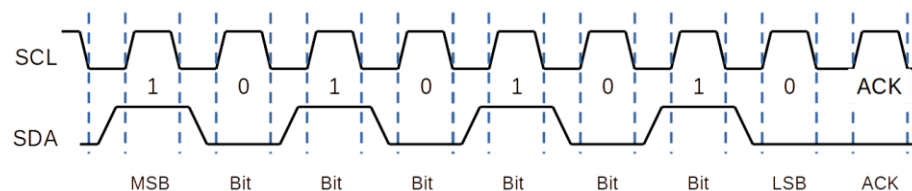
9. ábra - I²C érvényes adattovábbítás

Említettem pár gondolattal korábban a Start és Stop biteket. Ezek olyan “események”, amikre nem vonatkozik az, hogy az SDA láb szintje csak az SCL láb alacsony szintjén változhat. Sőt! A szabvány úgy írja elő ezt a két jelzést, hogy az adat lábon a fel- vagy lefutó élnek az SCL magas állapotában kell lennie. A 10. ábráról le tudjuk olvasni, hogy a Start bithez egy lefutó, a Stop bithez egy felfutó él szükséges.

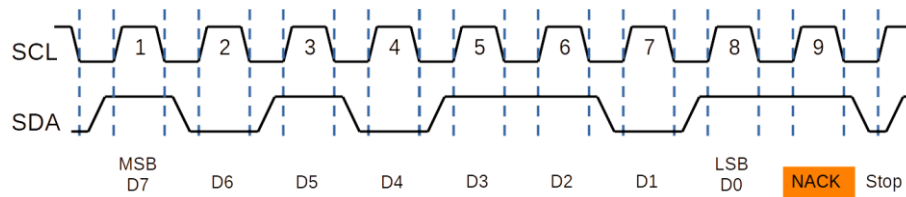


10. ábra - I²C Start és Stop bitek

Az ACK és NACK már ismert kifejezések, de nem tudjuk, pontosan mit is jelentenek. Most jött el az ideje annak, hogy ezen változtassunk. Az ACK és NACK is egy olyan bit, amit minden esetben az aktuálisan vevő eszköz fog küldeni az adónak. Az ACK esetén nullát, NACK esetén pedig egyet fog írni.

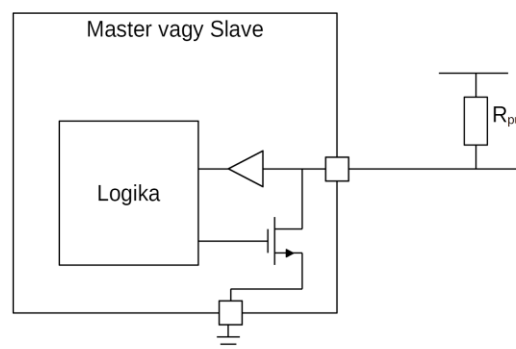


11. ábra - I²C üzenet ACK jelzéssel

12. ábra - I²C üzenet NACK jelzéssel

I²C PERIFÉRIA ELMÉLETI MEGVALÓSÍTÁSA

Eddig megismertük azt, hogyan néz ki a küldés és a fogadás, de azt még nem tudjuk, hogy ezt hogyan valósítjuk meg az eszközeinkben. A 13. ábrán az SDA és SCL lábak sematikus felépítését látjuk.

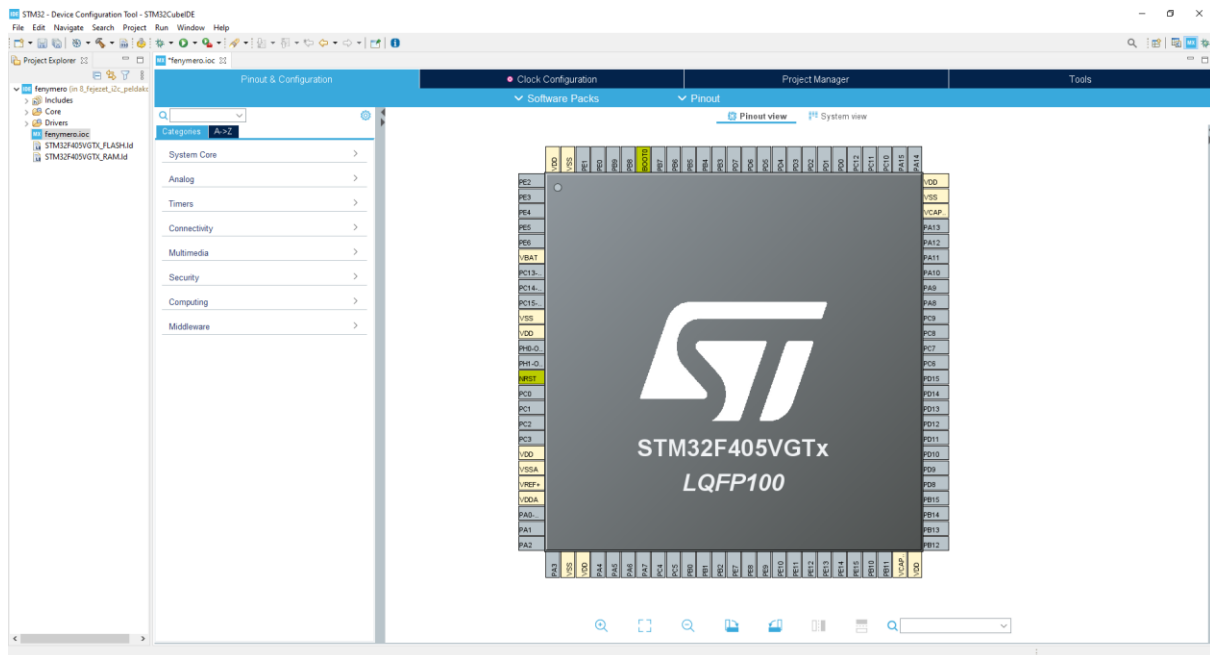
13. ábra - I²C eszközök egy SDA vagy SCL vonalának sematikus belső felépítése

A nagy négyzet jelöli az eszközt, ennek a határvonalán szereplő kis négyzetek jelölik a külvilághoz való csatlakozási pontokat. Az látjuk, hogy az IC tartalmaz egy "Logika" nevű logikai vezérlő blokkot, ami segítségével kapcsolgatjuk a FET-et. A képen észrevehetjük, hogy a kimenet Open-Drain meghajtással rendelkezik, amiről a 7. tananyagrészen bővebben olvashattál. Az I²C buszon egy vonalra több eszköz is csatlakozik, amik tervezési hiba vagy meghibásodás esetén akár szembe is hajthatnak egymással. Mivel Open-Drain kimenet alkalmazásával nem idézhetünk elő zárlatot, ezért legrosszabb esetben is csak meghiúsul a kommunikáció, de nem hibásodik meg több alkatrész a buszon.

I²C PÉLDAPROGRAM ELŐKÉSZÍTÉSE AZ STM32CUBEIDE SZOFTVERBEN

De mit ér a tudás, ha nem használjuk fel? Készítsünk el egy egyszerű példaprogramot a tananyaghoz! Ehhez előfeltétel, hogy legyen a gépünkön telepített STM32CubeIDE.

Indítsuk el az STM32CubeIDE-t és hozzunk létre egy új projektet: *File / New / STM32 project*. A korábban már megismert módon válasszuk ki az STM32F405VGTx mikrovezérlőt és végezzük el a szükséges beállításokat! Ha eddig mindent jól csináltunk, akkor a 14. ábrán látható *Pin & Configuration* ablaknak kell minket fogadnia.



14. ábra - STM32CubeIDE a még üres Pin & Configuration ablak

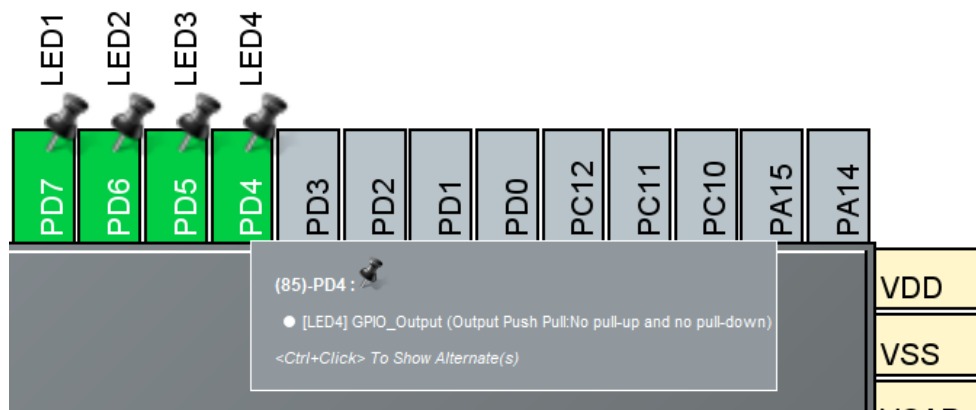
Itt tudjuk a mikrokontroller perifériáit engedélyezni, konfigurálni, a be- és kimenetként funkcionáló lábak működési módját meghatározni. A példánk során a panelünk négy darab LED-jét és a fénymérő szenzorát fogjuk használni, így elégséges csak a hozzájuk tartozó lábakat beállítani.

Először állítsuk is be a LED-eket vezérlőket.

LED neve	Melyik lábra van kötve
LED1	PD7
LED2	PD6
LED3	PD5
LED4	PD4

3. táblázat - LED-ek lábkiosztása a fejlesztői panelen

Ezeket a lábakat kimenetként fogjuk használni. Ehhez meg kell keresnünk, hogy hol vannak a programban. Vagy szemmel körbepfutjuk az összes lábat, vagy használjuk a láb kereső mezőt. Keressük meg a PD7-es lábat, majd kattintsunk rá és válasszuk ki a *GPIO_Output* lehetőséget. Ahhoz, hogy a kódunk könnyebben értelmezhető legyen, célszerű azt elnevezni. Kattintsunk *jobb egérgombbal* a PD7 lábra / *Enter User Label*. Írjuk be a LED nevét, ami a PD7-es láb esetén a LED1 (mivel ezt a LED-et vezérli). Ez a két lépést ismételjük meg a PD6, PD5, PD4 lábakra is!



15. ábra - STM32CubeIDE-ben a LED-ekhez tartozó lábak már beállítva és elnevezve

Következőnek állítsuk be az I²C perifériánkat, amit a fénymérő és a mikrovezérlő közötti kommunikációra fogunk használni. A mikrokontrollerek gyakran tartalmaznak ugyanolyan perifériából többet, ebben a controllerben például három I²C periféria kapott helyet, nekünk pedig az elsőre (I2C1) lesz szükségünk. Bal oldalon válasszuk ki ezt a *Connectivity* csoporton belül, majd a *Mode* felületen keressük meg az I2C-t! Ekkor a CubeMX hozzárendeli a perifériához a szükséges lábakat, ami a PB6 és PB7. Gyakran van lehetőségünk másik lábra kivezetni ugyanazokat a funkciókat. Például a PB6-os láb jelenleg az I2C1 SCL lába, azonban ezt a PB8-ra is ki tudnánk vezetni. A panelünkön az eredetileg is felajánlott PB6 és PB7 lábakra van huzalozva az I2C periféria 1-es csatornája, úgyhogy ezzel most nem kell foglalkoznunk.

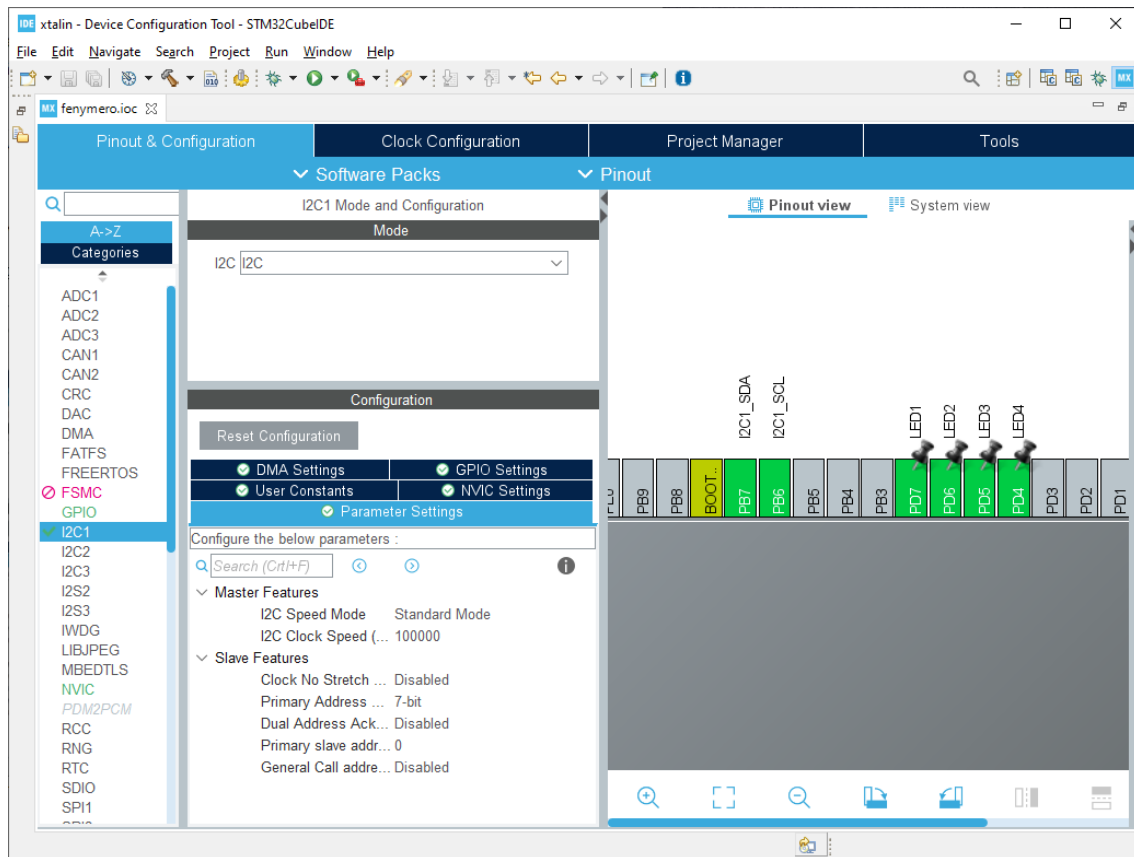
Az I²C szabvány a tananyag megírásának idején négyféle sebességi osztályt támogat. Ezek a standard mode (100 kbit/s), full speed (400 kbit/s), fast mode (1 Mbit/s) és high speed (3,2 Mbit/s) neveket viselik. Az általunk használt eszközök (mikrovezérlő és fénymérő) csak a standard és full speed módokat támogatják. A példa kedvéért jó lesz nekünk az alap standard mode és a hozzá tartozó 100 kbit/sec átviteli sebesség.

Léteznek olyan eszközök, amelyek 7 bitesnél hosszabb címeket használnak, mert az idők során a 7 bites címtartomány szűkösnek bizonyult. A *Primary Address Length* beállítás segítségével ennek hosszát állíthatjuk.

A *Clock Configuration* fülön tudjuk módosítani a controller órajelét. Én az alap 16 MHz-et hagytam a *HCLK*, *SYSCCLK* és az összes perifériamezőben.

A *Project Manager* fülön a *Code Generator* kategóriában keressük meg a *Generate peripheral initialization as a pair of '.c/.h' files per peripheral* lehetőséget. Ezzel a perifériákat inicializáló függvényeket külön fájlokba fogja írni, ami sokat segít a kód könnyebb olvashatóságán.

Ha végeztünk, mentsünk, és generáljuk le a kódot!

16. ábra - Kivezetések és I²C konfiguráció az CubeMX-ben

I²C PÉLDAPROGRAM

A programunk, amit ebben a tananyagrészen elkészítünk, legyen egy egyszerű kis progressbar (állapotcsík), amit a fénymérő szenzor megvilágításának függvényében tudunk vezérelni. Az elkészült projektet a tananyag mellékleteként találod meg.

```
int main(void)
{
    /* Összes periféria alapállapotba állítása, flash interfész és systick
    inicializálás */
    HAL_Init();

    /* Rendszerórajel konfigurálás */
    SystemClock_Config();

    /* Az összes konfigurált periféria inicializálása */
    MX_GPIO_Init();
    MX_I2C1_Init();

    // Kapcsoljuk be a fényerősség mérőt
```

```
Light_Sensor_Power_On(SIMPLE_MODE);

while (1)
{
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
        // Olvassuk ki a fénymérő által mért értéket
        uint16_t light_sensor_value = Light_Sensor_Read_Value(SIMPLE_MODE);

// Jelenítsük meg a kapott értéket a LED-eken a beállított limiteknek megfelelően
    if (light_sensor_value < LIMIT_OFF)
    {
        HAL_GPIO_WritePin(LED1_GPIO_Port, LED1_Pin, GPIO_PIN_RESET);
        HAL_GPIO_WritePin(LED2_GPIO_Port, LED2_Pin, GPIO_PIN_RESET);
        HAL_GPIO_WritePin(LED3_GPIO_Port, LED3_Pin, GPIO_PIN_RESET);
        HAL_GPIO_WritePin(LED4_GPIO_Port, LED4_Pin, GPIO_PIN_RESET);
    }
    else if (light_sensor_value < LIMIT_1)
    {
        HAL_GPIO_WritePin(LED1_GPIO_Port, LED1_Pin, GPIO_PIN_SET);
        HAL_GPIO_WritePin(LED2_GPIO_Port, LED2_Pin, GPIO_PIN_RESET);
        HAL_GPIO_WritePin(LED3_GPIO_Port, LED3_Pin, GPIO_PIN_RESET);
        HAL_GPIO_WritePin(LED4_GPIO_Port, LED4_Pin, GPIO_PIN_RESET);
    }
    else if (light_sensor_value < LIMIT_2)
    {
        HAL_GPIO_WritePin(LED1_GPIO_Port, LED1_Pin, GPIO_PIN_SET);
        HAL_GPIO_WritePin(LED2_GPIO_Port, LED2_Pin, GPIO_PIN_SET);
        HAL_GPIO_WritePin(LED3_GPIO_Port, LED3_Pin, GPIO_PIN_RESET);
        HAL_GPIO_WritePin(LED4_GPIO_Port, LED4_Pin, GPIO_PIN_RESET);
    }
    else if (light_sensor_value < LIMIT_3)
    {
```

```
        HAL_GPIO_WritePin(LED1_GPIO_Port, LED1_Pin, GPIO_PIN_SET);
        HAL_GPIO_WritePin(LED2_GPIO_Port, LED2_Pin, GPIO_PIN_SET);
        HAL_GPIO_WritePin(LED3_GPIO_Port, LED3_Pin, GPIO_PIN_SET);
        HAL_GPIO_WritePin(LED4_GPIO_Port, LED4_Pin, GPIO_PIN_RESET);
    }
    else
    {
        HAL_GPIO_WritePin(LED1_GPIO_Port, LED1_Pin, GPIO_PIN_SET);
        HAL_GPIO_WritePin(LED2_GPIO_Port, LED2_Pin, GPIO_PIN_SET);
        HAL_GPIO_WritePin(LED3_GPIO_Port, LED3_Pin, GPIO_PIN_SET);
        HAL_GPIO_WritePin(LED4_GPIO_Port, LED4_Pin, GPIO_PIN_SET);
    }
}
```

Kezdjük el az értelmezést a kályhánál, majd haladjunk tőle egyre távolabb! A programunk belépési pontja a `main()` függvény (mely a `main.c` fájlban található). Nézzük meg, hogy mi is történik ott.

Nagy vonalakban az alábbi dolgok zajlanak le benne:

1. Inicializáljuk a HAL library-t
2. Inicializáljuk az órajeleket
3. Inicializáljuk a GPIO és I²C perifériákat
4. A fénymérőnek kiadjuk a Power On utasítást
5. Kiolvassuk a fénymérő által mért értéket (fő ciklusban)
6. Megjelenítjük azt a LED csíkon (fő ciklusban)

Az első három dolgot elvégzi nekünk a kódgenerátor, viszont a többi már a mi dolgunk. Először ugorjunk a megjelenítésre, majd egy kicsit később visszatérünk az fénymérővel való kommunikációra.

Az érték, amit kapunk a fénymérőtől, egy előjel nélküli 16 bites szám (mivel nem létezik negatív megvilágítás), így nem kell foglalkozunk a nullánál kisebb értékek kezelésével.

```
// A LED-ek kikapcsolásának szintjei
const uint16_t LIMIT_OFF = 1000;
const uint16_t LIMIT_1 = 2000;
const uint16_t LIMIT_2 = 3000;
const uint16_t LIMIT_3 = 4000;
```

A main.c fájl elején lévő konstansokhoz fogjuk hasonlítani a mért értéket. Mindegyik konstans az adott tartomány végét jelöli, tehát az az érték, ami a LIMIT_OFF alatt van, az mind a négy LED-et kikapcsolja. Ami előlött de a LIMIT_1 alatt van, az csak az első LED-et kapcsolja be és így tovább a LIMIT_3-ig. A LIMIT_3 értékénél nagyobb vagy egyenlő számok hatására már az if-else szerkezet else ága fog lefutni, mely az összes LED-et bekapcsolja. Ezeket a limiteket a main elején a saját igényeinknek megfelelően át tudjuk állítani. Én ezekkel a paraméterekkel tudtam a legjobban elvégezni a szoftver tesztelését.

```
void Error_Handler(void) // Hibakezelő függvény
{
    /* USER CODE BEGIN Error_Handler_Debug */
        // Összes LED kikapcsolása
        HAL_GPIO_WritePin(LED1_GPIO_Port, LED1_Pin, GPIO_PIN_RESET);
        HAL_GPIO_WritePin(LED2_GPIO_Port, LED2_Pin, GPIO_PIN_RESET);
        HAL_GPIO_WritePin(LED3_GPIO_Port, LED3_Pin, GPIO_PIN_RESET);
        HAL_GPIO_WritePin(LED4_GPIO_Port, LED4_Pin, GPIO_PIN_RESET);

        // LED-ek villogtatása
        while (1)
        {
            HAL_GPIO_TogglePin(LED1_GPIO_Port, LED1_Pin);
            HAL_GPIO_TogglePin(LED2_GPIO_Port, LED2_Pin);
            HAL_GPIO_TogglePin(LED3_GPIO_Port, LED3_Pin);
            HAL_GPIO_TogglePin(LED4_GPIO_Port, LED4_Pin);

            HAL_Delay(200);
        }
    /* USER CODE END Error_Handler_Debug */
}
```

A main() függvény után láthatunk egy Error_Handler() nevű függvényt. Ezt a HAL library és a mi kódunk is használja, ha valamilyen hiba előfordul. Az volt a célom, hogy egyértelműen lássuk, hogy valami hiba történt, így a LED-eket először kikapcsoljuk (azért, hogy minden LED ugyanabba az állapotba kerüljön), majd 200 milliszekundumos időközönként villogtatjuk őket. Mivel ebben a hibajelzésben egy while (1) ciklus van, innen csak a mikrovezérlő újraindításával léphetünk ki.

A main() függvényben szereplő Light_Sensor_Read_Value() és Light_Sensor_Power_On() függvényt nem a main.c fájlban deklaráltuk, hanem a light_sensor.h fájlban. Ez segíti az

olvashatóságot, mivel nem egy hosszú main.c fájlunk van, hanem több kisebb, beszédes és könnyen értelmezhető modulra bontjuk szét a kódunkat.

```
// Ezt a két függvényt tudjuk használni a fénymérő bekapcsolására és az ADC0  
csatornájának az olvasására.  
  
// A függvények paraméterként kapják meg azt, hogy az előbbi két használati mód  
közül hogyan végezzék el a műveletet.  
  
void Light_Sensor_Power_On(enum I2C_USAGE_MODE mode);  
uint16_t Light_Sensor_Read_Value(enum I2C_USAGE_MODE mode);
```

Vegyük elő a light_sensor.h fájlt! Ebben három enumot hoztam létre. Ezek az I2C_LIGHT_SENSOR_REGISTERS, I2C_LIGHT_SENSOR_ADDRESS és az I2C_USAGE_MODE.

```
// A fénymérő regiszterei.  
// A szenzor adatlapja szerint a regiszter címéhez hozzá kell "vagyolni" 0x80-at.  
// Példa:  
// DATA0LOW regiszter adatlap szerinti címe: 0x0A  
// A ténylegesen írandó cím: 0x0A | 0x80, tehát 0x8A  
enum I2C_LIGHT_SENSOR_REGISTERS  
{  
    CONTROL = 0x80,  
    TIMING,  
    THRESHLOWLOW,  
    THRESHLOWHIGH,  
    THRESHHIGHLOW,  
    THRESHHIGHHIGH,  
    INTERRUPT,  
    ID = 0x8A,  
    DATA0LOW = 0x8C,  
    DATA0HIGH,  
    DATA1LOW,  
    DATA1HIGH  
};  
  
// A fénymérő lehetséges címei az ADDRSEL láb függvényében.  
// A HAL library úgy várja el a címet, hogy a bájt "tetején" helyezkedik el.
```

```
// Ehhez el kell tolni a 7 bites címet balra 1 bittel.
// Példa:
//      A szenzor adatlapja szerint, ha a lábat a földre kötjük a 7 bites címe:
0b00101001
//      A ténylegesen felhasználandó cím: 0b00101001 << 1, tehát 0b01010010
enum I2C_LIGHT_SENSOR_ADDRESS
{
    ADDR_SEL_ON_GROUND = 0x52,
    ADDR_SEL_FLOATS = 0x72,
    ADDR_SEL_ON_VDD = 0x92,
};

// A példakódban két módon fogjuk használni a HAL library-t.
// A SIMPLE_MODE azt fogja megmutatni, hogyan kell a fejezetben megismert
szekvenciát kódként megfogalmazni.
// A MEMORY_MODE segítségével el tudjuk fedni a szekvenciát, rövidebb kódot tudunk
írni és a kód olvashatósága is javul.
enum I2C_USAGE_MODE
{
    SIMPLE_MODE,
    MEMORY_MODE
};
```

Az I2C_LIGHT_SENSOR_REGISTERS nevű enum a fényszensor regisztereinek címét tartalmazza. Ha jobban megnézed, akkor ezek a regisztercímek nem egyeznek meg az 1. táblázatban található címekkel, mivel ezt az IC-t a megszokottól egy kicsit eltérően kell kezelni. Például, ha az *ID regiszterét* szeretném olvasni, azt nem simán megmondjuk neki, hogy “na akkor menjünk a 0x0A regiszteredre”, hanem azt kell mondanunk, hogy “COMMAND módban menjünk a 0x0A regiszteredre”. Ez elsőre nekem is érdekes volt, úgyhogy ezt most rögtön ki is fejtem. Az adatlap szerint, ha szeretnék egy regisztert “piszkálni” azt úgy tudjuk megtenni, hogy a regiszter írásakor a címhez még hozzáadunk 0x80-at. Tehát a regiszter címezésekor a 0x8A címet kell mondanunk a fénymérőnek.

Az I2C_LIGHT_SENSOR_ADDRESS enum a fénymérő lehetséges I²C címeit tartalmazza. Korábban már említettem, hogy a címezésnél a bájt felső hét bitje lesz a slave címe, az utolsó bit pedig az adatáramlás iránya. Az utolsó bitet a HAL library a használt függvényeinknek megfelelően saját magától állítgatja, nekünk csak annyi a feladatunk, hogy a 7 bites cím kezdetét felvigyük az MSB-re. Ehhez balra kell shiftelni a címet eggyel (meg kell szorozni 2¹-nel, tehát kettővel).

Az I2C_USAGE_MODE enum csak azért készült, mivel kétféleképpen írtam meg a Power_On() és Read_Value() függvényeket. A main.c fájlban, amikor meghívjuk őket, ott tudjuk meghatározni, hogy milyen módon szeretnénk használni a függvényt.

A három enumon kívül még a Light_Sensor_Power_On() és Light_Sensor_Read_Value() függvények prototípusát tartalmazza ez a fájl.

```
// A light_sensor.h fájlban deklarált függvény implementációja.  
// Paraméterként megkapott módon fogja végrehajtani a CONTROL regiszterbe való  
// POWER_ON érték írását.  
  
void Light_Sensor_Power_On(enum I2C_USAGE_MODE mode)  
{  
    HAL_StatusTypeDef write_status = HAL_ERROR;  
  
    switch (mode)  
    {  
        case SIMPLE_MODE:  
            write_status = Light_Sensor_Power_On_Simple();  
            break;  
        case MEMORY_MODE:  
            write_status = Light_Sensor_Power_On_Memory();  
            break;  
    }  
    // Hibakezelés, ha nem sikeres az érték írás  
    if (write_status != HAL_OK)  
    {  
        Error_Handler();  
    }  
}  
  
// A light_sensor.h fájlban deklarált függvény implementációja.  
// Paraméterként megkapott módon fogja végrehajtani a DATA0LOW és DATA0HIGH  
// regiszterekből való több bájtos olvasást.  
  
uint16_t Light_Sensor_Read_Value(enum I2C_USAGE_MODE mode)  
{  
    HAL_StatusTypeDef read_status = HAL_ERROR;
```

```
uint16_t light_sensor_value = 0;
switch (mode)
{
case SIMPLE_MODE:
    read_status = Light_Sensor_Read_Value_Simple(&light_sensor_value);
    break;
case MEMORY_MODE:
    read_status = Light_Sensor_Read_Value_Memory(&light_sensor_value);
    break;
}
// Hibakezelés, ha nem sikeres az olvasás
if (read_status != HAL_OK)
{
    Error_Handler();
}

return light_sensor_value;
}
```

Jól haladunk! Már csak egy fájl van hátra. Keressük is meg a light_sensor.c fájlt!

Ebben a fájlban valósítjuk meg az előbb említett két függvényt. Mindkét függvény hasonlóan működik. Paraméterként megkapják, hogy milyen módon szeretnénk elvégezni a műveletet, majd egy switch-case szerkezetben a módnak megfelelő, a tényleges műveletet elvégző függvényt hívjuk meg. A mode paraméter azt mondja meg, hogy milyen módon szeretnénk elvégezni a műveletet. A Simple és Memory végű függvények HAL függvényeket rejtenek magukban, és ezek visszatérési értékét adják vissza az őket meghívóknak, akik sikertelen művelet esetén ezt jelzik nekünk azzal, hogy belépnek a korábban megismert Error_Handler() függvénybe.

Folytassuk az átnézés Light_Sensor_Power_On_Simple() és Light_Sensor_Read_Value_Simple() függvényekkel. Már tudjuk azt, hogy íráskor és olvasáskor is meg kell mondanunk az eszköz címét, az adatirányt és a regisztert, amivel a műveletet végezzük. Ez történik itt is. A HAL_I2C_Master_Transmit() függvénynek meg kell adni azt, hogy melyik I²C perifériát, milyen I²C címet, honnan (pointer a küldendő adatra), hány bájtot küldjön és hogy mekkora timeout-ot (időkorlátot) engedünk neki a küldés végrehajtására.

```
// A light_sensor.h fájlban ismerttetett módok közül ez a függvény fogja az írási szekvenciát bemutatni.
```

```
HAL_StatusTypeDef Light_Sensor_Power_On_Simple()
```

```

{
    uint8_t bytes_to_send[2] = { CONTROL, POWER_ON };    //    küldeni    kívánt
    adatok megadása

    HAL_StatusTypeDef write_status = HAL_I2C_Master_Transmit(&hi2c1,
    LIGHT_SENSOR_ADDRESS, &bytes_to_send[0], 2, 100);        // adatküldés

    //az adatküldés sikerességének megfelelő visszatérés

    return write_status;
}

```

A `Light_Sensor_Power_On_Simple()` függvényben két bájtot fogunk küldeni, mivel a slave eszköz címét a HAL függvény magától elküldi. A két bájtunk egyike a regiszter címe (`CONTROL`) lesz, a másik pedig a `POWER_ON` konstans. Erre a küldésre azért van szükségünk, mivel a fénymérőn a mérést engedélyeznünk kell. Az adatlap szerint ezt úgy tudjuk megtenni, hogy a `CONTROL` regiszterbe beírjuk a `POWER_ON` néven tárolt értéket.

```

// A light_sensor.h fájlban ismertetett módok közül ez a függvény fogja elrejteni
az írási szekvenciát.

HAL_StatusTypeDef Light_Sensor_Power_On_Memory()
{
    uint8_t control_reg_content = POWER_ON; // regiszterbe betölteni kívánt adat
    // regiszter írás

    HAL_StatusTypeDef write_status = HAL_I2C_Mem_Write(&hi2c1,
        LIGHT_SENSOR_ADDRESS, CONTROL, I2C_MEMADD_SIZE_8BIT,
        &control_reg_content, 1, 100);

    //az adatküldés sikerességének megfelelő visszatérés

    return write_status;
}

```

A `Light_Sensor_Power_On_Memory()` függvénnyel egy kicsit magasabb szintről kezelhetjük a perifériánkat. Itt is meg kell adnunk azt, hogy melyik I²C-t használjuk, milyen címre küldünk, milyen regisztert írunk és hogy mit írunk. A fő különbség az, hogy itt nem kell a küldendő adataink közé felvenni a regisztert, mivel azt a küldő HAL függvény egy másik paramétereként adjuk meg.

A fénymérő már elkezdte a mérést, nekünk csak ki kell olvasni az eredményt. Erre ad megoldást a `Light_Sensor_Read_Value_Simple()` és `Light_Sensor_Read_Value_Memory()` függvény.

Nézzük meg először a `Light_Sensor_Read_Value_Simple()` nevűt. Itt nekünk kell megvalósítani a korábban látott I²C írási szekvenciát.

```

// A light_sensor.h fájlban ismertetett módok közül ez a függvény fogja bemutatni
a több bájtos olvasási szekvenciát.

HAL_StatusTypeDef Light_Sensor_Read_Value_Simple(uint16_t *output_value)

```

```

{

    // küldeni kívánt adat, jelen esetben az olvasni kívánt regiszter
    uint8_t byte_to_send = { DATA0LOW };

    HAL_StatusTypeDef write_status = HAL_ERROR;

    write_status = HAL_I2C_Master_Transmit(&hi2c1, LIGHT_SENSOR_ADDRESS,
    &byte_to_send, 1, 100); //adatküldés

    //az adatküldés sikerességének megfelelő visszatérés
    if(write_status != HAL_OK) {
        return write_status;
    }

    // tömb létrehozása a kiolvasandó adatoknak
    uint8_t register_content[2] = { 0 };

    HAL_StatusTypeDef read_status = HAL_I2C_Master_Receive(&hi2c1,
    LIGHT_SENSOR_ADDRESS, &register_content[0], 2, 100); //adat fogadás

    //tömb értékeinek összeadása egy változóba
    *output_value = register_content[1] << 8 | register_content[0];

    return read_status;
}

```

Meg kell először oldanunk az eszköz regiszterének kiválasztását. Ehhez használunk egy HAL_I2C_Master_Transmit() hívást, ahol megmondjuk a regiszter címét, de adatot nem küldünk bele. Ezután megkezdhetjük az olvasást a HAL_I2C_Master_Receive() függvénnyel. Ennek a függvénynek az íráshoz hasonlóan meg kell adnunk az I²C perifériát, a slave címet, egy pointert (ami a fogadott adat tárolásának helyére mutat), egy számot, hogy hány bájtot olvasson és végül egy timeout-ot. A kapott adatot, ami jelenleg még két darab 8 bites számként van tárolva, át kell alakítanunk egy 16 bites számmá. Ezt a számot a Light_Sensor_Read_Value_Simple() függvény a paramétereként megkapott memóriaterületre fogja menteni és visszatérési értéként az olvasás sikerességét adja.

```

// A light_sensor.h fájlban ismertetett módok közül ez a függvény fogja elrejteni
a több bájtos olvasási szekvenciát.

HAL_StatusTypeDef Light_Sensor_Read_Value_Memory(uint16_t *output_value)
{

```

```
// tömb létrehozása a kiolvasandó adatoknak
uint8_t register_content[2] = { 0 };

// memória olvasás a megadott DATA0LOW címről
HAL_StatusTypeDef read_status = HAL_I2C_Mem_Read(&hi2c1,
          LIGHT_SENSOR_ADDRESS, DATA0LOW, I2C_MEMADD_SIZE_8BIT,
          &register_content[0], 2, 100);

//tömb értékeinek összeadása egy változóba
*output_value = register_content[1] << 8 | register_content[0];

return read_status;
}
```

A másik olvasófüggvényünk (`Light_Sensor_Read_Value_Memory()`) használata során nem kell foglalkozunk a szekvencia betartásával, mivel ezt a `HAL_I2C_Mem_Read()` megoldja. Nekünk csak meg kell adnunk a korábban már említett paramétereket és hátra is dőlhetünk.

A példakód a fénymérő ADC0 adatát olvassa ki. Ha szeretnéd kipróbálni a programunk működését, akkor keresd meg a panelen a *Light sensor* felíratú alkatrészt és világítsd meg valamilyen fényforrással! Ha a fényforrás és a szenzor közé rakod a kezed úgy, hogy a fénynek egyre nagyobb részét takarod ki, akkor szépen lassan elkezd fogyni a világító LED-ek a száma.

Úgy készült el a példakód, hogy könnyen lehessen az alábbi paramétereket változtatni:

1. Fénymérő I²C címét (ha nem a földre kötöd az ADDRSEL lábat)
2. A LED-ek kikapcsolásának limitjeit
3. A fénymérő bekapcsolásának és
4. A fénymérő olvasásának módját

Biztatlak arra, hogy kísérletezz ezekkel vagy akár olvasd ki a másik adatregisztert. Attól fejlődünk, hogy próbálkozunk!

SPI

Az SPI (Serial Peripheral Interface) kommunikációt csak elméleti szinten fogjuk érinteni ebben a tananyagrészen, mivel a fejlesztői panel nem tartalmaz olyan szenzort, ami ezt a kommunikációs interfészt használja.

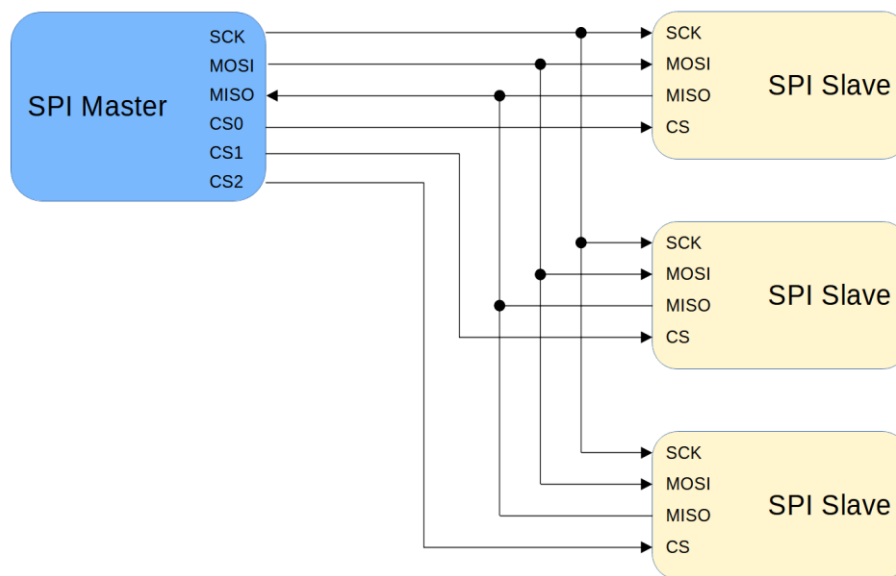
Nézzük meg, miben hasonlít egymásra a két (I²C és SPI) protokoll.

- Az SPI szintén egy master-slave kommunikáció.
- Mindkettő szinkron átvitelen alapul.

- A master itt is több slave eszközzel lehet összekötve

A fenti hasonlóságok ellenére elég sok tulajdonságukban különböznek.

- Az I²C esetében a címzés egy adatbájt segítségével történik, míg az SPI-nál a master a slave eszköznek egy dedikált lábát, az SS-t (Slave Select) állítja. Ezt a lábát szokták még CS (Chip Select) lábnak is hívni. (17. ábra)
- Az SPI esetén kommunikáció lehet full-duplex is (mindkét irányban egyszerre van adatáramlás).
- SPI esetén nincs a szabványban előre meghatározott adatátviteli sebesség.



17. ábra - SPI busz felépítése

LÉPTETŐREGISZTEREK

Az SPI perifériák működésének alapjai a léptetőregiszterek (shift register). Ezek olyan áramkörök, amik a bennük lévő adatot minden órajel hatására egy bit pozícióval arrébb mozgatják (léptetik). A bemeneti adatot bitenként órajel hatására a shift register 0. bitjébe töltjük, a korábbi 0. bit értékét az 1. bit helyére toljuk és így tovább. Ez így megy végig a teljes regiszteren, az utolsó helyiértéken lévő bit "kiesik" a regiszterből. Az alábbi ábrán egy négy bites shift register működését láthatod, ahol minden sor egy léptetési ciklust jelent.

Bemenet	0. bit	1. bit	2. bit	3. bit	Kimenet
0b11010011	0	0	0	0	0b0
0b1101001	1	0	0	0	0b00
0b110100	1	1	0	0	0b000
0b11010	0	1	1	0	0b0000

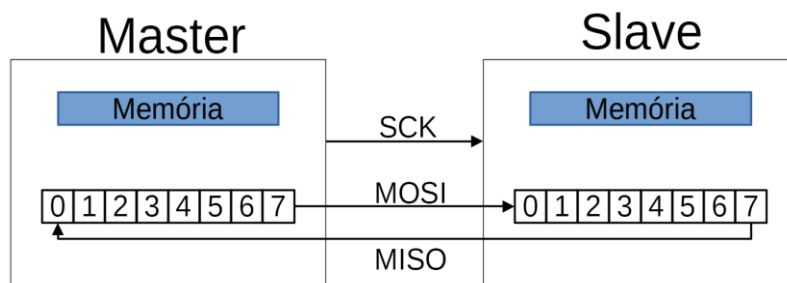
Bemenet	0. bit	1. bit	2. bit	3. bit	Kimenet
0b1101	0	0	1	1	0b00000
0b110	1	0	0	1	0b100000
0b11	0	1	0	0	0b1100000
0b1	1	0	1	0	0b01100000
0b0	1	1	0	1	0b00110000

4. táblázat - Léptetőregiszter példa

Az SPI MŰKÖDÉSE

Nézzük meg, hogy full-duplex módban hogyan működik az SPI!

Ahogy már mondtam, itt is meg kell címezni a slave eszközt. Ezt úgy tudjuk megtenni, hogy a master megfelelő választólábát (amelyik össze van kötve a slave SS lábával) logikai 0 szintre húzzuk. És itt jön a csavar az I²C-hez képest, mivel nincs kifejezetten írási és olvasási ciklus, ezért mindkét dolgot egyszerre csináljuk. Mindkét SPI eszköz (master és slave) tartalmaz egy-egy léptetőregisztert. A léptetőregiszterek bemenete a másik eszköz léptetőregiszterének kimenetéhez van kötve, így egy "gyűrűt" képeznek az adatoknak.



18. ábra - SPI, mint két léptetőregiszter

A kommunikáció során az átküldött adatok az egyik eszköz shift-regiszteréből indulnak, és a másikba érkeznek, tulajdonképpen helyet cserélnek a kommunikációs ciklus végére.

PÉLDA SPI ADATÁTVITELRE

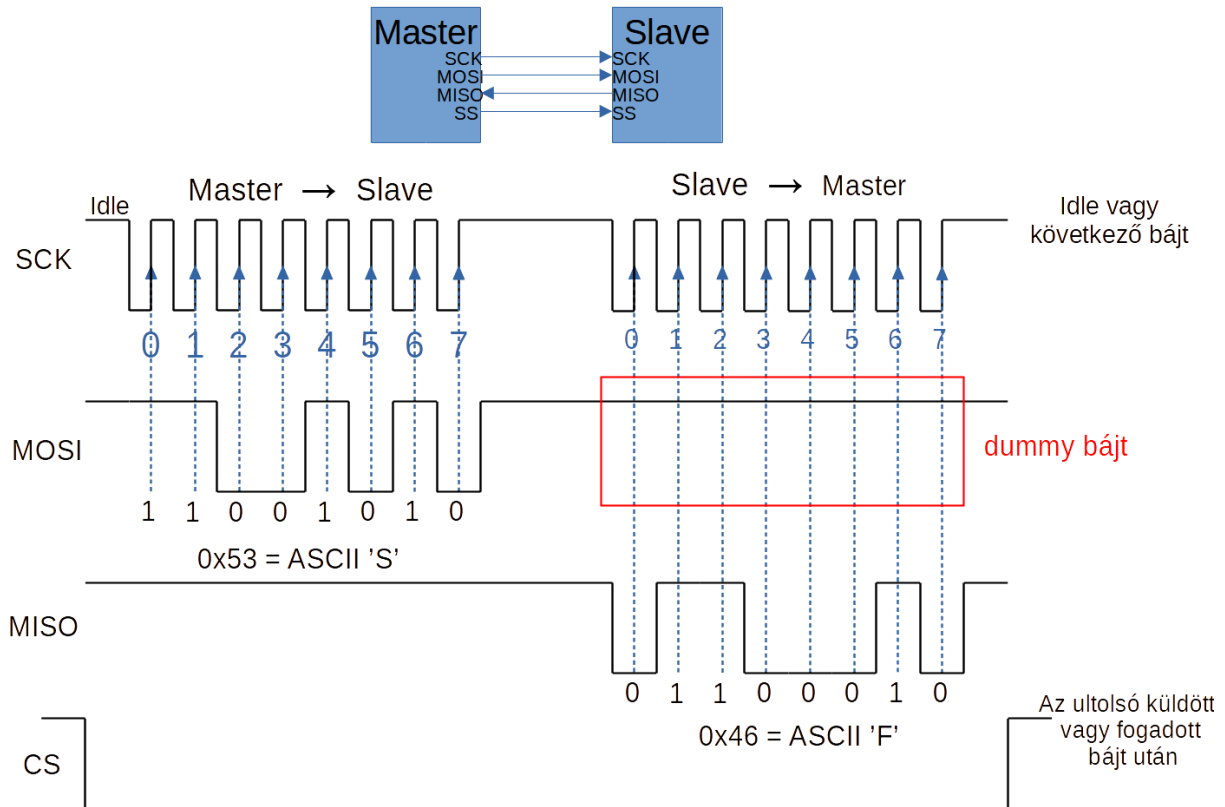
Folytassuk a korábban elkezdett példát!

Ehhez nézzük meg a 19. ábrán látható kommunikációt. A slave egység egy belső regiszterének értékét szeretnénk kiolvasni. Ezt a már ismert konvenció szerint egy cím azonosítja, amit először a slave tudomására kell hoznunk, majd utána fogadhatjuk a kívánt adatbájtot.

Feltételezzük azt, hogy a master a slave eszköz 0x53-as című belső regiszterét szeretné kiolvasni. Ekkor a következők történnek:

1. A master lehúzza a slave SS lábát földpontenciálra.

2. A master kiküldi az olvasandó regiszter címét.
3. Tart egy kis szünetet, hogy a slave-nek legyen ideje előkészíteni az adatát.
4. A master 0xFF bájtot (egy úgynevezett dummy bájtot) küld a slave-nek, amivel kicserélik a léptető regisztereik tartalmát.
5. Miután elfogytak a küldendő/olvasandó bájtok, a master már nem ad órajelet és az SS lábát logikai 1 szintre állítja.



19. ábra - SPI adatátvitel

A dummy bájtot azt a célt szolgálja, hogy legyen valami általunk definiált adat, amit a slave eszköznek tudunk adni a kívánt adatért cserébe. Az értéket gyakran 0xFF-ként vagy 0x00-ként határozzuk meg. Nem mindig van rá szükség, vannak olyan esetek, amelyekben elhagyhatjuk, ezt a felhasznált alkatrész adatlapjából megtudhatjuk.

ÖSSZEFOGLALÁS

A tananyagrészen megismerkedtünk a beágyazott környezetben leggyakrabban használatos szinkron kommunikációs interfészekkel. Érintettük a soros és párhuzamos kommunikáció közti különbségeket. Megismertük a két legelterjedtebb protokollt, az I²C-t és az SPI-t. A hardvertervezés során az egységes kommunikációs csatornák ismerete kritikusán fontos kérdés, hiszen a legtöbb szenzort és egyéb alkatrészt így tudjuk kezelni.